

LAG

Keep missing values and order meaningful

01 / THE SAS EXAMPLE

```
data test;
  input id value;
  prev_value = lag(value);
  prev2_value = lag2(value);
  change = value - prev_value;
  datalines;
1 10
2 15
3 12
4 18
5 20
;
run;
proc print data=test noobs;
run;
```

ATTACH SOURCE OR PASTE IT AFTER THE PROMPT. START FRESH EACH TIME.

WHAT YOU WILL PRACTICE

State what the model should preserve when there is no previous value.

Each LAG call executes on every row in this program. The supplied order defines the previous values. Empty starting positions mean no earlier observation; they remain missing.

EXPECTED RESULT

id	value	prev_value	prev2_value	change
1	10	.	.	.
2	15	10	.	5
3	12	15	10	-3
4	18	12	15	6
5	20	18	12	2

A period denotes SAS numeric missing; pandas may display NaN or <NA>. Compare missing positions as well as numbers. Conditional LAG calls require a separate analysis.

Expected values checked in Python; SAS execution pending.

Prompt A: leave choices open

High degrees of freedom / Notice the decisions the model makes for you.

COPY THIS PROMPT + THE SAS EXAMPLE

Convert this SAS code to Python.
Show code in this response. Do not create or run files, or read existing Python files.

LOOK AT THE RESPONSE

Which library and method did it choose? Did it preserve the expected behavior? Compare the result with page 1 before accepting it.

TRY IT / THREE FRESH ATTEMPTS

Run A three times with the same SAS example and setup. Save each first response before asking follow-up questions. Record agreement as well as differences.

Bedrock A/B trials for this example are pending.

WHY CHOICES REMAIN OPEN

- 1 A familiar relationship**
Previous-value calculations are common. The model may choose shift and give the correct answer immediately.
- 2 Choices remain open**
Library, method, and missing-value representation remain unstated. Different-looking code may preserve the same behavior.
- 3 Meaning needs protection**
Check missing positions and row order. Filling an unavailable previous value can create a change that was never observed.

ANTI-PATTERN TO AVOID

Anti-pattern: "Fill the blanks." Using zero for the first previous value manufactures a change of 10. Explain that missing means no prior observation and must remain missing.

Prompt B: make the choices explicit

Low degrees of freedom / State the behavior, then ask for a check.

COPY THIS PROMPT + THE SAME SAS EXAMPLE

Convert this SAS code to Python using pandas.
Show code in this response. Do not create or run files, or read existing Python files.

- Recreate id and value, preserving all five rows and their order.
- Each SAS LAG call executes on every row in this program.
- prev_value: use value.shift(1), with no frequency argument.
- prev2_value: use value.shift(2), with no frequency argument.
- change: subtract prev_value from value.
- Leave unavailable prior values and resulting changes missing. Do not fill them with zero, drop rows, sort, or wrap values.
- Include assertions for the original id sequence and these lists:
prev_value: [missing, 10, 15, 12, 18]
prev2_value: [missing, missing, 10, 15, 12]
change: [missing, 5, -3, 6, 2]
Use pd.isna to test missing positions.
- Return one complete script. Compute results from value; do not hard-code the result columns.

Briefly explain why the first change is missing.

WHY THESE DETAILS HELP

- 1

Protect the sequence
Preserving row order keeps each value connected to the right predecessor. Sorting would change those relationships.
- 2

Limit the equivalence
Per-row execution explains why shift fits this program. Other uses of SAS LAG need their own analysis.
- 3

Specify missing-value behavior
Explicit missing checks protect the empty starting positions. Meaning matters more than the printed missing-value symbol.

CHECK IT YOURSELF

Run the script separately. Check all numbers and missing positions. Row 3 must show -3. The first row must not acquire a wrapped final value or a change calculated from zero.

State what missing means and preserve the sequence that gives each row its context.